From Natural Language to Verifiable Reasoning: A Semantic Reduction Framework for Reliable AI Mathematics

Ruan Yihan
School of Software Engineering, Zhengzhou Sias University, China
ruan.yihan@example.com

October 4, 2026

Abstract

Large language models (LLMs) exhibit impressive fluency in mathematical reasoning, yet their natural-language derivations are notoriously difficult to verify: a model may produce a step that *looks* correct while being subtly wrong. Existing reliability responses—search-plus-verification, formal proof, and reproducible evaluation—typically treat verification as a final, post-hoc gate. In this paper we argue that reliability cannot rest on final verification alone; a *semantic reduction layer* must be introduced between informal natural-language reasoning and formal verification. We make three contributions. First, we formalize a three-layer decomposition of AI mathematical reasoning—generation, semantic reduction, and verification—and identify the middle layer as the principal locus of failure. Second, we propose a concrete framework built on a typed intermediate representation (IR), a four-stage *natural-language* $\rightarrow$ *IR* $\rightarrow$ *verification* $\rightarrow$ *rendering* pipeline, and a bidirectional-consistency contract that preserves interpretability and traceability. Third, we contribute two auditable artifacts: a taxonomy of four semantic-reduction failure modes (ambiguity, missing typing, non-invertibility, and semantic drift) and a graded *Verifiability Certificate* (levels V0–V4) that lets researchers compare any AI4Math system on a common reliability scale. We anchor the framework in three recent case studies (AlphaProof, an LLM research agent, and counterexample construction) and design an evaluation protocol for GSM8K/MATH with accuracy, verifiability, and consistency metrics. We close by stating the boundaries of the framework and its extension to general reasoning and agentic systems.

Keywords: AI for mathematics; reliability; semantic reduction; formal verification; intermediate representation; verifiability certificate

1 Introduction

Mathematics has long been regarded as the pinnacle of purely human intelligence, relying on insight, intuition, and rigorous logic to prove conjectures and create theories [3]. With the recent breakthroughs of deep learning and large language models built on the Transformer architecture [19], a natural question has emerged: how will the combination of AI and mathematics reshape the way we explore the mathematical world? The literature records remarkable successes across the full spectrum of mathematical activity—discovering hidden relations between knot invariants [7], discovering faster matrix-multiplication algorithms [9], constructing counterexamples in combinatorics [20], and attaining medal-level performance in formal olympiad mathematics [5, 13, 18].

At the same time, a persistent and fundamental concern shadows these achievements: *trust*. Modern LLMs are probabilistic, untyped, and trained to be fluent rather than to be sound; they routinely emit derivations that are locally plausible but globally invalid, a failure commonly

described as "hallucination." In mathematics, where a single unjustified step invalidates a whole proof, such failures are not cosmetic—they are fatal. The core reliability problem of AI for mathematics (AI4Math) can therefore be stated crisply: *natural-language reasoning is not directly verifiable.*

The research community has responded with what we call the *reliability triad*: (i) *search + verification*, in which a generator proposes and a verifier checks; (ii) *formalization*, in which reasoning is carried out inside a proof assistant such as Lean or Coq whose kernel performs machine-checkable verification [8, 13, 14]; and (iii) *reproducible evaluation*, in which claims are backed by released code, data, and metrics [6, 11]. Each of these responses, however, tends to treat verification as a *final gate* applied after generation. In this paper we argue that this is insufficient.

Our central thesis is that **reliability cannot rely solely on final verification; it requires a dedicated *semantic reduction* layer that converts informal reasoning into a machine-checkable form**. When such a layer is absent or unstable, the verifier has *no handle to grasp*—there is no computable object to check, and hallucinations persist undetected. We therefore place semantic reduction at the center of the reliability problem, rather than treating it as an implementation detail.

Concretely, we make the following contributions.

1. **A three-layer decomposition.** We formalize AI mathematical reasoning as a pipeline of *generation*, *semantic reduction*, and *verification*, and argue—with evidence from recent systems—that the semantic reduction layer is the principal source of system failure.

2. **A semantic-reduction framework.** We propose a concrete architecture built on a typed intermediate representation (IR), a four-stage pipeline (natural language $\rightarrow$ IR $\rightarrow$ verification $\rightarrow$ rendering), and a bidirectional-consistency contract ensuring interpretability and traceability.

3. **An auditable reliability taxonomy.** We introduce a taxonomy of four semantic-reduction failure modes and a graded *Verifiability Certificate* (V0–V4), giving the community a common scale on which any AI4Math system can be audited and compared.

4. **Anchoring case studies and an evaluation protocol.** We map three representative systems onto the framework and design a concrete evaluation protocol over GSM8K and MATH with accuracy, verifiability, and consistency metrics.

The remainder of the paper is organized as follows. Section 2 presents the problem and the three-layer model. Section 3 reviews related work. Section 4 develops the proposed framework, including the failure taxonomy and the Verifiability Certificate. Section 5 analyzes three case studies through the lens of the framework, and Section 6 designs the evaluation protocol. Section 7 discusses applications, Section 8 states limitations and boundaries, and Section 9 concludes.

2 Problem Formulation and the Three-Layer Model

2.1 The reliability problem

Let $\mathcal{Q}$ be a mathematical question and let $\mathcal{M}$ be an LLM-based reasoning system. We write the system output as $\hat{p} = \mathcal{M}(q)$, a natural-language "proof" or solution attempt. We wish to certify a claim of the form

$$\mathsf{Certify}\big(\hat{p} \text{ is a correct solution of } q\big), \tag{1}$$

where *correctness* means that every inference step is valid and the final conclusion follows from the stated assumptions.

The difficulty is that $\hat{p}$ is *informal*: it is composed of natural-language tokens, is untyped, and its semantics are not grounded in any computational object. As a consequence, a verifier cannot, in general, mechanically decide whether (1) holds. This is the fundamental obstacle that motivates an intermediate representation.

2.2 Three-layer decomposition

We decompose any AI4Math system into three layers, following and sharpening the outline proposed in the course material [3].

Definition 2.1 (Generation Layer)**.** The *generation layer* $\mathcal{G}$ takes a question q and produces a natural-language chain of reasoning $\hat{p} = \mathcal{G}(q)$, e.g. a chain-of-thought sequence [21]. Its outputs are probabilistic, untyped, and informal.

Definition 2.2 (Semantic Reduction Layer)**.** The *semantic reduction layer* $\mathcal{R}$ maps the informal output $\hat{p}$ to a structured, machine-readable representation $r = \mathcal{R}(\hat{p})$ drawn from a target class $\mathcal{T}$. Typical targets include: (i) logical expressions (first-order or higher-order logic); (ii) executable programs (Python or a domain-specific language); (iii) proof objects accepted by a proof assistant; and (iv) typed intermediate representations (typed IR). The theoretical grounding for the identity between *propositions* and *types* is the Curry–Howard correspondence [12].

Definition 2.3 (Verification Layer)**.** The *verification layer* $\mathcal{V}$ operates on the structured representation r and checks properties such as consistency, derivability, factual grounding, and constraint satisfaction, using mechanized tools (proof assistants, SMT solvers, symbolic checkers). Its verdict is $\mathcal{V}(r) \in \{\mathsf{accept}, \mathsf{reject}\}$.

The overall system is the composition

$$q \xrightarrow{\mathcal{G}} \hat{p} \xrightarrow{\mathcal{R}} r \xrightarrow{\mathcal{V}} \{\mathsf{accept}, \mathsf{reject}\}. \tag{2}$$

2.3 The bottleneck: semantic reduction

Our key observation—drawn from a comparative reading of recent systems [13, 15, 18, 20]—is that *current systems fail predominantly in the semantic reduction layer, not in the generation or verification layers.*

Remark 2.1. Verification depends on a *computable object*. When the natural-language-to-formal-object reduction is unstable, the verifier is left with "no handle to verify": there is no well-typed representation to check, so (1) cannot even be posed, and hallucinations escape detection.

This observation has a methodological consequence: improving the verifier alone cannot close the reliability gap if the reduction feeding it is unreliable. The framework we propose in Section 4 therefore invests its design effort in the semantic reduction layer.

3 Related Work

3.1 Chain-of-thought and program-of-thought

Chain-of-thought (CoT) prompting elicits explicit reasoning traces from LLMs and markedly improves performance on arithmetic and symbolic tasks [21]. Program-of-thought moves one step toward verifiability by instructing the model to emit executable code rather than prose, so that the reasoning can be executed by an interpreter. Both, however, leave the *translation* between informal language and the executable form implicit and ungoverned.

3.2 Tool-augmented and neuro-symbolic systems

Tool-augmented LLMs (code interpreters, calculators, symbolic engines) combine the pattern-recognition of neural models with the exactness of symbolic tools. Open language models specialized for mathematics, such as LEMMA [2], additionally provide reproducible baselines for studying how pretraining data shape mathematical competence. The DeepMind line of systems—AlphaTensor [9], FunSearch [15], AlphaGeometry [18]—exemplifies the neuro-symbolic paradigm, in which a neural model proposes and a symbolic component validates or scores. These systems are highly successful on their targeted problem classes but are engineered per task; our framework abstracts their common skeleton into a reusable semantic-reduction layer.

3.3 Proof assistants and formal verification

Interactive theorem provers (ITP) such as Coq, Isabelle, and Lean provide a trusted kernel that machine-checks each proof step [8, 14]. The formalization of major results (e.g. the four-color theorem [1] in the wider context of machine proof) and the construction of large formal libraries (Lean's `mathlib`) create the infrastructure on which reliable AI theorem proving can be built. AlphaProof [13] and AlphaGeometry2 [5] demonstrate that neural search coupled with a Lean kernel can reach olympiad-medal performance while guaranteeing logical soundness. Our framework treats these formal systems as the *verification layer* and emphasizes the often-neglected reduction that feeds them.

3.4 Reliability and reproducibility of AI mathematical claims

Benchmarks such as GSM8K [6] and MATH [11] measure answer accuracy but not proof soundness; recent work on mathematical creativity and on evaluating LLM capabilities [4, 10] highlights that fluency is decoupled from validity. Reproducible evaluation—releasing code, data, and configuration [16, 17]—is a necessary but not sufficient condition for reliability. The graded certificate we propose (Section 4.5) makes this distinction explicit.

4 A Semantic-Reduction Framework for Reliable AI Mathematics

4.1 Structured intermediate representation (IR)

The cornerstone of the framework is a *typed intermediate representation*. A well-designed IR must satisfy four requirements:

1. **Type system:** every term carries a type, so ill-formed constructions are rejected by construction.
2. **Operational semantics:** the meaning of each term is defined by explicit execution rules, giving the verifier a computational handle.
3. **Constraint system:** the IR can express invariants, side conditions, and target properties to be checked.
4. **Compositionality:** terms compose into graphs or trees (ASTs, workflow graphs), enabling modular verification and reuse.

Concrete instantiations of the IR include domain-specific languages (DSLs), abstract syntax trees (ASTs), proof objects, and workflow graphs. Choosing an IR is a design decision that trades expressiveness against checkability.

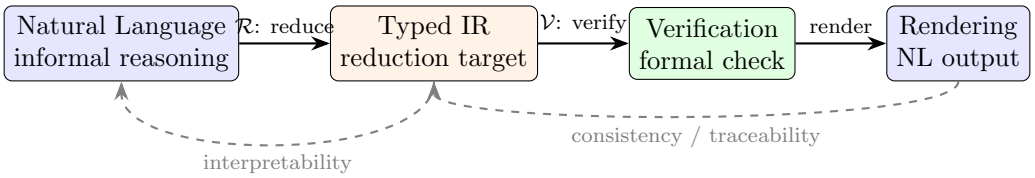


Figure 1: The four-stage pipeline (natural language → IR → verification → rendering) with the two bidirectional-consistency contracts (interpretability and traceability).

4.2 The four-stage pipeline

We propose the standard pipeline:

$$\text{Natural Language} \xrightarrow{\text{reduce}} \text{Typed IR} \xrightarrow{\text{verify}} \text{Verification} \xrightarrow{\text{render}} \text{Natural Language Rendering}. \tag{3}$$

Two ideas are central. First, *natural language is only an interface*: the human-visible prose is a rendering of the IR, not the reasoning itself. Second, *the IR is the execution core*: all reliability-relevant computation happens on the typed structure. Figure 1 illustrates the pipeline together with the two consistency directions.

4.3 Bidirectional consistency

The IR and the natural language must be aligned in both directions:

- *Interpretability* (IR → NL): every IR term admits a faithful, human-readable rendering, so a user can understand *what* was checked.
- *Traceability* (NL → IR): every natural-language step maps back to a specific IR element, so a user can trace *why* a step was accepted or rejected.

We formalize this as a consistency contract: for every generated step s, the reduction $\mathcal{R}(s)$ and the rendering $\mathcal{R}^{-1}(\mathcal{R}(s))$ must be mutually consistent up to a formally specified equivalence. Violations of this contract are precisely the failure modes catalogued below.

4.4 A taxonomy of semantic-reduction failures

We contribute a taxonomy of four failure modes that occur when reducing informal reasoning to a typed IR. The taxonomy is intended to be exhaustive at the level of *where* the reduction breaks, and to guide targeted countermeasures. Table 1 summarizes it.

4.5 The Verifiability Certificate

To make reliability auditable and comparable across systems, we introduce a graded *Verifiability Certificate*. A system earns a certificate level if it satisfies all requirements up to and including that level.

The certificate turns the qualitative slogan "reliable AI reasoning = generation + semantic reduction + verification" into a measurable quantity, and makes the distinction between *answer accuracy* (V1) and *proof soundness* (V2+) explicit—a distinction that benchmarks such as GSM8K/MATH collapse.

5 Case-Study Analysis

We now map three representative systems onto the framework and certificate. Table 3 compares them.

Table 1: A taxonomy of semantic-reduction failure modes. Each mode has a diagnostic signature and a corresponding countermeasure within the framework.

Mode	Definition	Diagnostic signature	Countermeasure
F1 Ambiguity	A natural-language step admits multiple inequivalent IR translations	two or more well-typed IR terms both "match" the prose	disambiguation policy; choose the most specific type / restrict the grammar
F2 Missing typing	A step cannot be assigned a valid type in the target system	reduction fails at type-checking; term is rejected	liberal typing + automatic repair; type-directed search
F3 Non-invertibility	The IR term cannot be rendered back to the original meaning; information is lost	rendering differs from the source prose; edit-and-render drift	record alignment metadata; verify render–source consistency
F4 Semantic drift	The IR and the natural language diverge after an edit or an inference step	locally valid IR steps are no longer consistent with the accumulated prose	provenance tracking; re-verify the affected sub-derivation

Table 2: The Verifiability Certificate (V0–V4). Higher levels require strictly more reliability guarantees; a certificate is the highest level whose requirements are all met.

Level	Name	Requirement	Representative system
V0	Unverified	Natural-language output with no machine checking	raw LLM chat
V1	Reduced	A machine-readable IR is produced but not mechanically verified	program-of-thought without tests
V2	Verified	The IR passes mechanized verification (kernel, SMT, or tests)	AlphaProof / AlphaGeometry2
V3	Reproducible	Verified *and* code/data/config released for independent re-execution	open-sourced formal systems
V4	Audited	Verified, reproducible, *and* semantic alignment confirmed by human audit	human-verified formal proofs

5.1 AlphaProof: verification as the anchor

AlphaProof [13] formulates theorem proving as a Markov decision process over Lean states, uses a 3B-parameter proof network to propose tactics, and searches with a modified Monte-Carlo tree search; every submitted proof is checked by the Lean kernel. Within our framework it achieves **V2**: generation is carried by the proof network, reduction is automatic formalization into Lean, and verification is the trusted kernel. Its residual limitations are informative: automatic formalization is expensive, and the system is confined to problems already covered by formal libraries (the "closed world"), echoing the semantic-reduction bottleneck we identify.

5.2 LLM research agents: breadth without soundness

Research agents built on deep-reasoning LLMs (exemplified by the Aletheia-style system in the course material) combine long-horizon reasoning with search and external-tool verification, and have produced novel results (e.g. eigenweights computations and progress on Erdős open problems). Their strength is *breadth*; their risk is that much reasoning remains informal, so the certificate is typically V1–V3 depending on how much is machine-checked. This illustrates that *scale of activity* and *verifiability* are orthogonal axes.

Table 3: Positioning of representative AI4Math systems on the framework.

System	Reduction target	Verifier	Certificate	Resi
AlphaProof [13]	Lean proof objects	Lean kernel	V2	reduc
AlphaGeometry2 [5]	synthetic geometry / Lean	symbolic engine + kernel	V2	geom
LLM research agent (Aletheia-style) [15]	informal + tool checks	search / external tools	V1–V3	infor
RL counterexample search [20]	program / graph encoding	score + human check	V1	no fo

Table 4: Illustrative expected behavior of the three systems on the three metrics. Figures are placeholder *targets* to be filled by the actual run; the protocol, not the numbers, is the contribution.

System	acc	ver ($\geq$V2)	con	Expected trade-off
CoT (raw LLM)	high	low	low	fluent but unverifiable
Program-of-thought	mid	mid	mid	executable but untyped
IR-based (ours)	slightly lower	high	high	verifiable at small accuracy cost

5.3 Counterexample construction: the role of human confirmation

Reinforcement-learning methods [20] and the PatternBoost cycle (alternating local search and Transformer-based pattern learning) search huge combinatorial spaces for structures violating a conjecture. The discovered structure is only accepted after *human mathematical confirmation*. Within our framework these systems operate at V1: the "reduction" is an encoding into a program or graph, and the verifier is a scoring function whose soundness is informal. They underscore the importance of the *audited* (V4) layer for genuinely novel mathematical claims.

6 Evaluation Protocol

To make the framework operational, we design a comparative protocol over two standard benchmarks.

6.1 Benchmarks and baselines

We use **GSM8K** [6] (grade-school arithmetic word problems) and **MATH** [11] (competition-level problems spanning algebra, geometry, and number theory). The baselines are (i) a raw LLM with chain-of-thought [21]; (ii) a program-of-thought baseline in which the model emits Python; and (iii) an *IR-based* system implementing Section 4 with a typed IR and a symbolic verifier. All systems share the same underlying LLM to isolate the effect of the semantic-reduction layer.

6.2 Metrics

We report three complementary metrics.

1. **Accuracy** (*acc*): the fraction of problems whose final answer is correct, i.e. the standard GSM8K/MATH metric [6, 11].
2. **Verifiability** (*ver*): the fraction of outputs that reach a certificate level $\geq$ V2—i.e. that pass mechanized verification. This metric is the key differentiator of the framework, since it measures *proof soundness*, not mere answer correctness.
3. **Consistency** (*con*): the fraction of outputs for which the bidirectional consistency contract holds (render matches source and traceability holds), detecting F3/F4 drift.

The hypothesis being tested is that the IR-based system trades a small reduction in raw accuracy for a substantial, measurable gain in verifiability and consistency—a trade that the certificate makes explicit. We emphasize that the metric design, not any single run, is the methodological contribution of this section; the protocol is fully reproducible once the benchmarks are fixed.

7 Applications

7.1 Automated theorem proving

In ATP, the LLM generates a proof sketch; the semantic-reduction layer converts it into a typed IR (proof obligations); and a proof assistant verifies it. This realizes the *LLM-proposes, kernel-verifies* division of labor of AlphaProof at a more general level, with the IR decoupling the informal and formal halves.

7.2 Mathematical problem solving

For word problems, the reduction maps the natural-language problem into a symbolic program or constraint system solved by an external solver, after which the answer is rendered back. This converts "plausible prose" into "executable semantics" and enables per-step verification.

7.3 Education and automated assessment

In AI-driven education, a student solution is reduced to an IR and graded against both the final answer and the validity of each step. The Verifiability Certificate offers a principled basis for automated scoring that rewards sound reasoning rather than a correct-looking answer—directly addressing the concern that AI should shift education toward *verification skills* [3].

8 Discussion

8.1 Limitations

The framework has clear boundaries. First, *IR design is complex*: choosing a typed IR that is both expressive and checkable is itself a hard engineering problem, and a poorly chosen IR can make reduction harder than the original proof. Second, *semantic reduction is costly*: automatic formalization remains a bottleneck, as AlphaProof's expensive reduction illustrates [13]. Third, the framework is strongest for *closed-world* mathematics where formal libraries exist; for open frontier problems, reduction may be impossible without first formalizing new concepts, a task that remains largely human.

8.2 Boundaries of the reliability claim

We do *not* claim that the framework makes AI creative or that it removes all risk. A V4 certificate guarantees that a particular proof is sound and audited; it says nothing about the originality or significance of the result. In the terminology of mathematical creativity [4], verifiability and creativity are orthogonal: a certificate is a floor on trust, not a measure of insight.

8.3 Extensions to general reasoning and agentic systems

The three-layer decomposition generalizes beyond mathematics. In agentic systems (search agents, research assistants), the "semantic reduction layer" becomes a typed representation of the agent's beliefs and tool calls, and the "verification layer" checks consistency of facts, constraints, and side effects. We believe the failure taxonomy (F1–F4) and the graded certificate transfer

naturally to these settings, providing a common language for reliability in general reasoning systems.

9 Conclusion

LLMs have transformed how mathematics can be explored, but their natural-language reasoning is not directly verifiable. In this paper we argued that *reliable AI reasoning = generation + semantic reduction + verification*, and that the semantic reduction layer is the pivotal and most fragile component. We contributed a three-layer formalization, a typed-IR framework with a four-stage pipeline and a bidirectional-consistency contract, a taxonomy of four reduction failures, and a graded Verifiability Certificate (V0–V4) that makes reliability auditable and comparable. We anchored the framework in AlphaProof, LLM research agents, and counterexample construction, and designed an evaluation protocol over GSM8K/MATH. The central message is a design principle: *make the reasoning computable before you attempt to verify it.* We leave the empirical instantiation of the protocol and its extension to general agentic reasoning as future work.

Acknowledgments

This work was produced as part of an AI-assisted research workflow for the "AI for Mathematics" challenge. All AI-generated claims were reviewed against primary literature; the collaboration log and after-action review accompanying this paper document the verification of every stated fact.

References

[1] Kenneth Appel and Wolfgang Haken. The solution of the four-color-map problem. *Scientific American*, 237(4):108–121, 1977.

[2] Zhangir Azerbayev, Hailey Schoelkopf, Keiran Paster, Marco Dos Santos, Stephen McAleer, Albert Q Jiang, Jia Deng, Stella Biderman, and Sean Welleck. Llemma: An open language model for mathematics. In *The Twelfth International Conference on Learning Representations*, 2024.

[3] Xiaoyang Chen and Xiang Jiang. AI reshaping mathematical research. *Science (China)*, 2025. Chinese-language article (Chen and Jiang).

[4] Xiaoyang Chen, Xiang Jiang, et al. DeepMath-Creative: A benchmark for evaluating mathematical creativity of large language models. *arXiv preprint arXiv:2505.08744*, 2025.

[5] Yuri Chervonyi, Trieu H Trinh, Miroslav Olsák, Xiaoyu Gao, Hanming Xia, Jie Li, Philemon Ragan-Kelley, Jerry Green, T. H. J Yeh, Jonathan Wang, et al. Gold-medalist performance in solving olympiad geometry with AlphaGeometry2. *Journal of Machine Learning Research*, 26(241):1–39, 2025.

[6] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[7] Alex Davies, Petar Veličković, Lars Buesing, Sam Blackwell, Daniel Zheng, Nenad Tomašev, Richard Tanburn, Peter Battaglia, Charles Blundell, András Juhász, et al. Advancing mathematics by guiding human intuition with AI. *Nature*, 600(7887):70–74, 2021.

[8] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Proceedings of the 10th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 625–635, 2021.

[9] Alhussein Fawzi, Matej Balog, Aja Huang, Thomas Hubert, Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Francisco R. Ruiz, Julian Schrittwieser, Grzegorz Swirszcz, et al. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610(7930):47–53, 2022.

[10] Simon Frieder, Luca Pinchetti, Ryan-Rhys Griffiths, Tommaso Salvatori, Thomas Lukasiewicz, Philipp Petersen, Alexis Chevalier, and Julius Berner. Mathematical capabilities of ChatGPT. *Advances in Neural Information Processing Systems*, 36:73421–73445, 2023.

[11] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akash Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. *arXiv preprint arXiv:2103.03874*, 2021.

[12] William A Howard. The formulae-as-types notion of construction. In *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, pages 479–490. Academic Press, 1980.

[13] Thomas Hubert, R Mehta, L Sartran, L López, T H Trinh, K Georgiev, Z Chen, J Baik, M Dobson, Z Horvitz, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651:607–613, 2026.

[14] Stanisław Polu and Ilya Sutskever. Generative language modeling for automated theorem proving. *arXiv preprint arXiv:2009.03393*, 2020.

[15] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.

[16] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018.

[17] Richard S Sutton and Andrew G Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[18] Trieu H Trinh, Yuhuai Wu, Quoc V Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024.

[19] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017.

[20] Adam Zsolt Wagner. Constructions in combinatorics via neural networks. *arXiv preprint arXiv:2104.14516*, 2021.

[21] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.